TypeScript Senior Architect CheatSheet

Guía de Ingeniería de Tipos: Meta-Programming, Conditional Types, Template Literals, & Type System Internals

TS 5.X SPEC & MODERN PATTERNS

1. Type Inference, Conditional Types & Conditional Pattern Matching (infer)

Extracción de Tipos con `infer` & Tuple Unpacking

```
// Extrae el tipo de retorno de una función asíncrona
type UnwrapPromise<T> = T extends Promise<infer R> ? Unwr
apPromise<R> : T;

// Extrae el primer elemento y el resto de una tupla
type Head<T extends any[]> = T extends [infer H, ...infer
_] ? H : never;
type Tail<T extends any[]> = T extends [infer _, ...infer
R] ? R : [];

// Inferir argumentos de funciones sobrecargadas /
complejas
type FirstArg<T> = T extends (...args: [infer A, ...any[]]
) => any ? A : never;
```

Distribución de Tipos Condicionales & NonNullable Custom

```
// Los tipos condicionales se distribuyen automáticamente
sobre uniones
type ToArray<T> = T extends any ? T[] : never;
type StrOrNumArray = ToArray<string |
number>; // string[] | number[]

// Evitar la distribución envolviendo en tuplas [T]
type NonDistributiveToArray<T> = [T] extends [any] ? T[] :
never;
type CombinedArray = NonDistributiveToArray<string | numb
er>; // (string | number)[]
```

2. Meta-Programming: Template Literals & Mapped Types Remapping

Mapped Types & Key Remapping (`as keyof`)

```
// Genera Getters dinámicos fuertemente tipados
type Getters<T> = {
  [K in keyof T as `get${Capitalize<string & K>}`]: () =>
T[K];
};

// Filtrado dinámico de claves de un objeto por tipo de
valor
type FilterKeysByValue<T, V> = {
  [K in keyof T as T[K] extends V ? K : never]: T[K];
};

type User = { id: number; name: string; age: number };
type OnlyNumbers = FilterKeysByValue<User, number>; //
{ id: number; age: number; }
```

Deep Readonly & Deep Partial Recursivos

```
// Inmutabilidad profunda recursiva para estado global /
DDD
type DeepReadonly<T> = T extends (...args: any[]) => any
  ? T
  : T extends object
  ? { readonly [K in keyof T]: DeepReadonly<T[K]> }
  : T;

// Nested Partial para parches de API DTOs
type DeepPartial<T> = {
  [P in keyof T]?: T[P] extends object ? DeepPartial<T[P]>
: T[P];
};
```

3. Nominal Typing (Branded Types) & Type Guard Predicates

Nominal Typing / Branded Types (Type Safety)

```
// Previene mezclar primitivos estructurales (ej. UserId
vs OrderId)
declare const BrandSymbol: unique symbol;
type Brand<K, T> = K & { [BrandSymbol]: T };

type UserId = Brand<string, "UserId">;
type OrderId = Brand<string, "OrderId">;

const makeUserId = (id: string) => id as UserId;
const makeOrderId = (id: string) => id as OrderId;

let u: UserId = makeUserId("usr_123");
// u = makeOrderId("ord_999"); // ❌ ERROR DE COMPILACIÓN
DETECTADO
```

Custom Type Guards & Assertion Functions

```
// Custom Type Guard Discriminado
function isApiResponse<T>(res: unknown): res is { data: T;
status: number } {
  return (
    typeof res === "object" &&
    res !== null &&
    "data" in res &&
    "status" in res
  );
}

// Assertion Signatures para Invariantes (Lanza Error si
Falla)
function assertIsString(val: unknown): asserts val is str
ing {
  if (typeof val !== "string") throw new Error("Not a
string");
}
```

4. Built-in Utility Types Internals & Performance Optimization

Matriz de Built-in Utility Types de Referencia

```
// Reconstrucción interna conceptual de las utilities
nativas
```

tsconfig.json Senior Flags para Monorepos y Strictness

```
{
  "compilerOptions": {
```

```typescript
type MyOmit<T, K extends keyof any> = Pick<T, Exclude<keyof T, K>>;
type MyReturnType<T extends (...args: any) => any> = T extends (...args: any) => infer R ? R : any;
type MyParameters<T extends (...args: any) => any> = T extends (...args: infer P) => any ? P : never;
type MyAwaited<T> = T extends null | undefined ? T : T extends object && T extends { then(onfulfilled: infer F): any } ? F extends ((value: infer V, ...args: any) => any) ? MyAwaited<V> : never : T;
```

```jsonc
    "strict": true,
    "exactOptionalPropertyTypes": true, /* Evita undefined implícito en opcionales */
    "noUncheckedIndexedAccess": true, /* Trata obj[key] como T | undefined */
    "noImplicitOverride": true, /* Obliga override en Clases OOP */
    "target": "ES2022",
    "moduleResolution": "NodeNext",
    "skipLibCheck": true, /* Performance masivo en tsc build */
    "incremental": true
  }
}
```